

Projet de Rattrapage : Simulation d'un espace à N-Corps

1. Introduction.

Problème : Notre objectif principal au cours de ce projet était de simuler l'attraction gravitationnelle mutuelle de plusieurs corps dans l'espace. Au début il y a peu de corps, la charge de calcul pour la machine n'est pas trop élevée et plutôt rapide mais dès que le nombre de planète s'agrandit, si on garde une méthode naïve ou chaque corps interagit avec tout les autres les calculs sont de plus en plus nombreux a en devenir disproportionné et lents empêchant une machine normale de simuler ce pseudo univers.

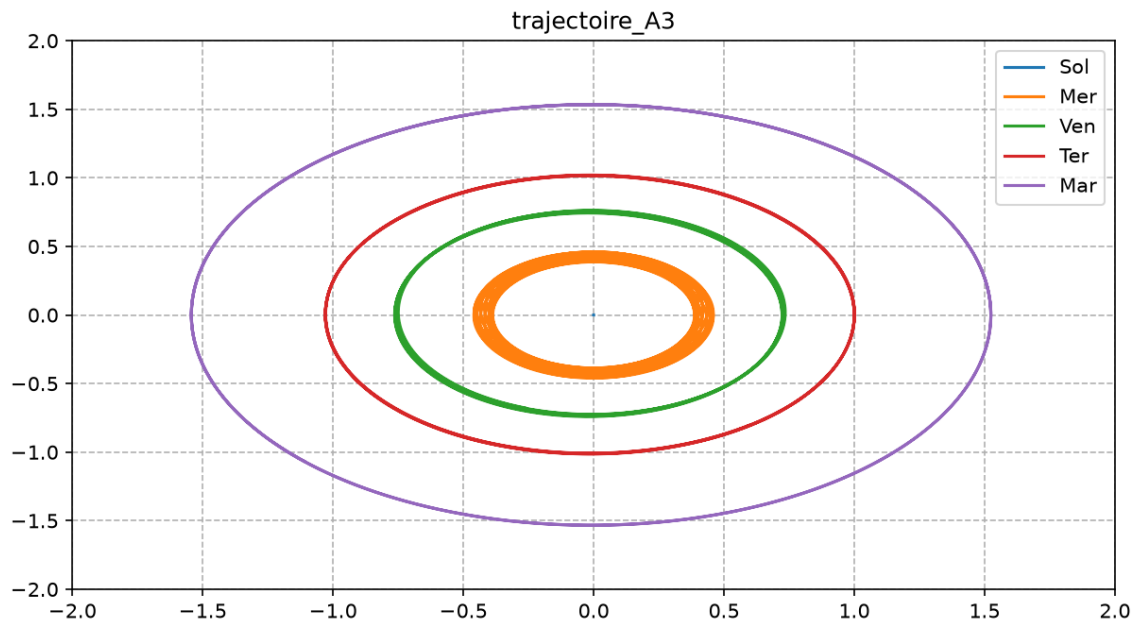
Objectif : Le but de ce projet est décomposé en plusieurs étapes. Dans un premier temps, il s'agit de construire et de valider un moteur physique stable en commençant avec des cas simples : deux corps dont un seul en mouvement, puis plusieurs corps en mouvement avec une attraction mutuelle et des caractéristiques différentes. Suite à la validation de ce socle physique, la seconde grande étape consiste à implémenter l'algorithme d'approximation spatiale de Barnes-Hut. Cette optimisation a pour but de réduire la puissance de calcul nécessaire et de simplifier le fonctionnement pour simuler des milliers de particules. Et pour finir rendre le tout visuel afin de pouvoir visualiser le fonctionnement et comparer différentes méthodes avec les données tirées de la/des simulation(s)

2. Validation du moteur.

- a. Pour commencer, nous avons dû implémenter le moteur de base au fonctionnement de ce simulateur. Il nous a été expliqué quel algorithme mathématique utiliser afin que notre projet soit physiquement stable sur le long terme. C'est pour cela que nous avons utilisé l'algorithme Leapfrog à la place de la méthode d'Euler.

L'explication de ce choix est très compréhensible car il nous est impossible de garder une valeur exacte lors d'une simulation sur machine, il faudra l'arrondir pour ne pas avoir de problèmes ou d'erreur. Or cet arrondissement est la faille principale a la méthode d'Euler qui marche très bien théoriquement et physiquement. Chaque calcul on risque une approximation or des milliers de calculs se font ce qui cumule une erreur qui a la fin devient colossale biaisant nos les résultats poussant les planètes à s'éloigner de leur trajectoire. Leapfrog est fait spécifiquement pour ce type de problème, en décalant le calcul de la vitesse et de la position d'un demi-pas, il va corriger le biais de l'approximation tout au long de la simulation nous permettant une simulation stable au long terme.

- b. Grace a ce moteur de base, nous avons pu simuler les trajectoires de certaines planètes du système solaire. Comme le graphique ci-dessous montre, le code simule correctement les orbites réels formant des ellipses stables autour du soleil. Quelques déformations sont visibles notamment sur mercure ou venus.



3. Optimisation de Barnes-Hut.

a. Le Quadtree :

Après avoir construit un modèle plus solide-il, nous fallait l'optimiser car en exécutant le programme avec la méthode de force brut on remarquait que le temps d'exécution était plutôt long en augmentant le nombre de corps.

Pour cette raison nous avons implémenter de Quadtree.

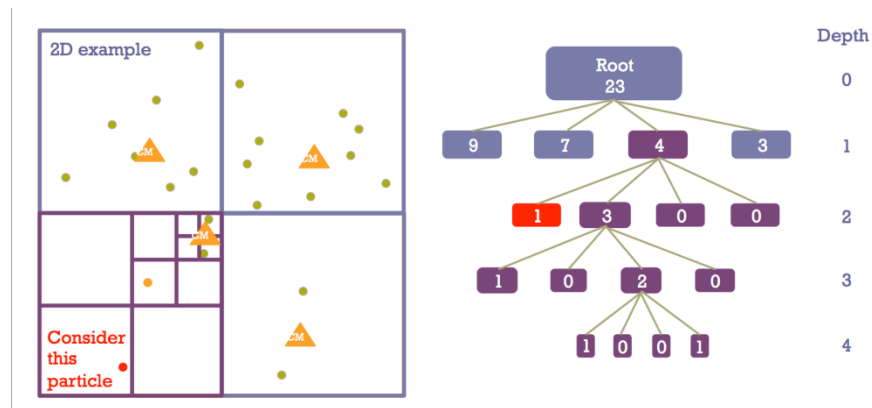
Son modèle est assez simple et visuel quand on le décompose en plusieurs étapes.

Premièrement l'algorithme découpe l'espace de façon récursive afin d'isoler chaque planète dans des zones carrées finale nommée une « feuille ». Pour faire plus simple si le grand carré contient plusieurs planètes il se découpe en 4 puis petits carrés.

Dans chaque carré on enregistre sa masse, et son centre de masse (l'ensemble des planètes qu'il contient) en remontant de carré en carré.

Et pour finir, au lieu de calculer l'attraction de chaque planètes de manière individuelle l'une après l'autre on regarde la distance et si un ensemble de planètes est loin on ne prend que le carré qui les englobe et on calcul la force globale comme si c'était une grosse planète.

En voici la représentation schématique :



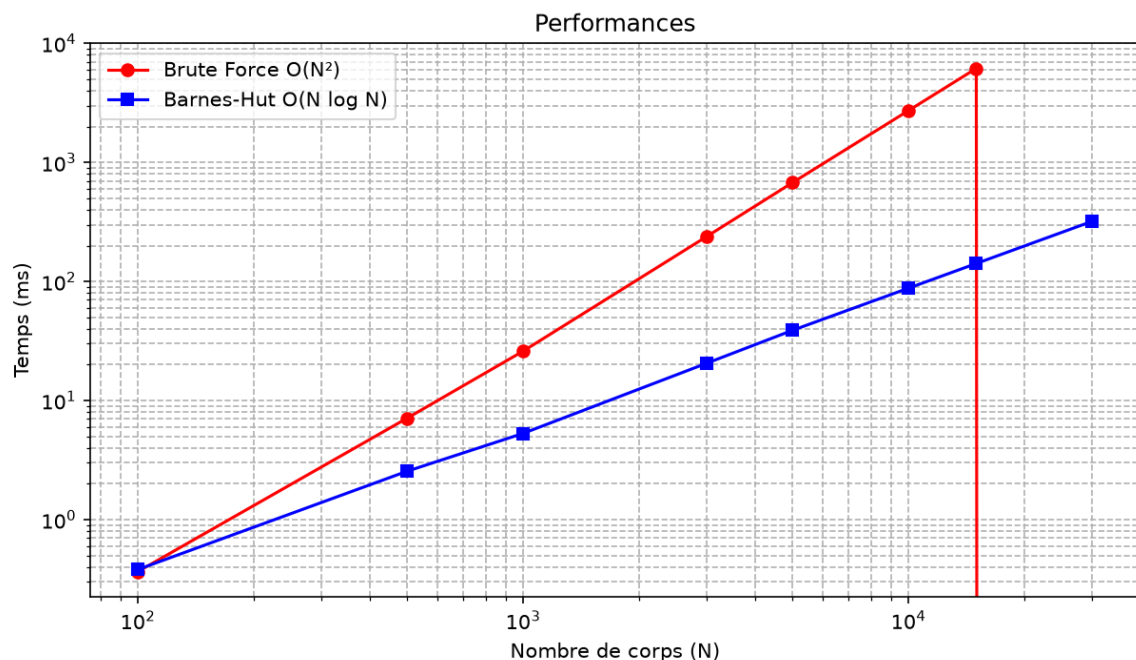
- b. Pour que cette approximation fonctionne, il nous fallait choisir un critère (thêta) afin de définir mathématiquement à partir de quelle distance un groupe de planètes est considéré comme "lointain". Ce paramètre dicte le compromis entre la vitesse de la simulation et la précision de la physique. Et grâce à nos simulation on a pu observer le pourcentage d'erreur face au thêta.

Thêta	Pourcentage d'erreur
0	$5.2e^{-14}\%$
0.15	1.19%
0.80*	5.33%
1	8.59%
2	70.7%

* Thêta personnel calculé avec la formule donnée

4. Performances.

Pour pouvoir comparer les deux méthodes nous avons réaliser un graphique sur une échelle Log-Log (logarithmique) qui nous permet de comprendre la complexité de celle-ci. En voici le graphique ci-dessous.



- Courbe Rouge | Force Brute : Sur le graphique on peut observer une pente de presque 2, ce qui démontre bien en échelle logarithmique une complexité $O(N^2)$ Si le nombre de corps est multiplié par 10 le temps lui est environ multiplié par 100.
- Courbe Bleue | Barnes-Hut : On observe une pente plus douce que la rouge avec un rapport un peu supérieur à 1. 1 représentant une complexité de N pour être un peu supérieur il faut donc rajouter le $\log(N)$ nous donnant une complexité $O(N(\log(N)))$
- On observe qu'au-dessus de 100 corps la force brut prend un tout autre chemin que la méthode de Barnes-Hut avec une pente bien plus inclinée.

5. Décomposition du projet.

Au cours de ce projet, ne sachant pas quel librairies j'allais utiliser et voulant découvrir un nouveau langage j'ai choisi le C++ ce langage est assez pratique pour sa rapidité d'exécution sa ressemblance a d'autres langages de programmation et la variété de librairies offertes. Et aussi du python pour l'implémentation facile d'un moyen d'afficher les graphiques.

Pour que tout soit clair j'ai créé des dossier afin de trier les fichiers en fonction de leurs utilité.

La représentation brève se trouve a la page d'après.

N-Body-Simulator/

```
├── 2D/
│   ├── src/          # Fichiers source C++
│   ├── headers/      # Fichiers headers
│   ├── csv/          # Fichiers de sortie
│   ├── bin/          # Exécutables compilés
│   └── Makefile       # Règles de compilation
├── traceur.py         # Script de visualisation
└── README.md          # Ce fichier
```

- 2D : Ce dossier servait au cas où j'avais le temps de faire la partie 3D bonus sinon tous les autres dossiers auraient pu remonter d'un niveau.
- src : Ce dossier contient toutes les parties source du programme, le fonctionnement général bien séparé par partie.
- Headers : contient tous les fichiers Header afin de permettre l'appel des fonctions implémentées de manière facile dans tous les autres programmes.
- Csv : Stocke tous les fichiers csv des mesures faites au cours des simulations plus les graphes.
- Bin : Contient tous les fichiers exécutables après avoir tout compilé.
- Makefile : Sert à l'adaptabilité de l'application, grâce à lui tout est facile d'exécution.
- Traceur.py : Ce fichier sert à tracer tous les graphes nécessaires à l'étude des algorithmes.
- README : Qui explique clairement comment lancer des simulations de manière visuelles ou globales et toutes les commandes ou informations utiles à propos du projet.

Quelques recherches ont dû être faites pour bien comprendre le sujet malgré que le sujet de base ait été très complet.

L'IA étant autorisée elle a eu sa présence dans le projet notamment pour m'apprendre les bases du projet et du langage de programmation, effectuer des recherches ou encore écrire les parties les plus complexe ou longue (e.g. réécrire tout les chemins de chaque fichiers, Makefile complet...)

6. Problèmes rencontrés.

Plusieurs problèmes ont été rencontrés tout au long du projet, en voici quelques exemples :

- Des erreurs au début a cause du compilateur, n'ayant pas installé un compilateur assez complet pour le langage, plusieurs problèmes sont apparus dès le début, ceux-ci ont rapidement été résolu avec un changement de compilateur.
- Un petit nombre de débordements, (stack overflow) avec les valeurs qui n'étaient pas arrondis.
- Des cas extrêmes rares de division par 0, ce ci majoritairement observé lors des teste de changement de variables, quand deux planètes se superposent tout le programme plante. Pour éviter cela certains critères ont été mis en place et aussi les valeurs les mieux adaptées.
- Problème très classique, l'oublie de téléchargement de librairie.